Institute of Business Administration, Karachi

Department of Computer Science

The 0/1 Knapsack Problem

Algorithm Analysis, Dynamic Programming & Real-World Applications

CSE 317: Design and Analysis of Algorithms

Spring 2026

Instructors

Dr. Shahid Hussain | Dr. Taslim Murad

Submitted by

Name	**Student ID**
Team Member 1	XXXX-XXXX
Team Member 2	XXXX-XXXX
Team Member 3	XXXX-XXXX
Team Member 4	XXXX-XXXX

Submission Date: Friday, 15 May 2026

Language: Python 3 · Algorithms: Brute Force · Memoization · Tabulation · Space-Optimised DP · Greedy Approximation

Contents

1 Introduction **2**
1.1 Problem Description . . . 2
1.2 Motivation and Real-World Relevance . . . 2
1.3 Formal Mathematical Formulation . . . 2
1.4 Input / Output Specification . . . 3
1.5 Computational Complexity Class . . . 3

2 Algorithm Description **3**
2.1 Overview of Approaches . . . 3
2.2 Approach 1 – Brute Force (Exhaustive Recursion) . . . 4
2.3 Approach 2 – Memoization (Top-Down DP) . . . 4
2.4 Approach 3 – Tabulation (Bottom-Up DP) . . . 5
2.5 Approach 4 – Space-Optimised DP (1-D Array) . . . 7
2.6 Approach 5 – Greedy Approximation . . . 7

3 Theoretical Analysis **9**
3.1 Formal DP Recurrence . . . 9
3.2 Dynamic Programming Prerequisites . . . 9
3.2.1 Overlapping Subproblems . . . 9
3.2.2 Optimal Substructure . . . 9
3.2.3 Why DP Applies . . . 9
3.3 Complexity Analysis . . . 10
3.4 Greedy Approximation Ratio . . . 10
3.5 Base Cases and Boundary Conditions . . . 10

4 Implementation Details **11**
4.1 Machine Specification . . . 11
4.2 Project Structure . . . 11
4.3 Test Case Sources . . . 11
4.3.1 Baseline Instance . . . 11
4.3.2 Benchmark Instances — kplib . . . 12
4.3.3 Input Generation Strategy . . . 12

5 Results and Discussion **12**
5.1 Correctness Verification — Baseline Instance . . . 12
5.2 Empirical Runtime Comparison — kplib Benchmarks . . . 13
5.3 Theoretical vs. Empirical Growth . . . 13
5.4 Greedy Approximation Quality by Instance Type . . . 14
5.5 Strengths and Weaknesses . . . 14

6 Conclusion **14**

1 Introduction

1.1 Problem Description

The **0/1 Knapsack Problem** is a classical combinatorial optimisation problem in computer science and operations research. Given a set of n items, each with a weight w_i and a value v_i, and a knapsack with a fixed weight capacity W, the objective is to select a subset of items such that the total weight does not exceed W and the total value is maximised.

The *0/1* designation reflects the defining constraint: each item must either be taken in its entirety (1) or left behind entirely (0). Fractional inclusion is strictly prohibited. This single restriction transforms the problem from a trivially solvable greedy task (the fractional variant) into an NP-Hard combinatorial challenge.

1.2 Motivation and Real-World Relevance

The knapsack model — binary selection under a single resource constraint with a maximisation objective — appears across a remarkably wide range of real-world domains:

- **Budget allocation:** selecting investment projects within a fixed capital budget where each project is either fully funded or rejected.
- **Cargo loading:** packing freight into an aircraft or truck to maximise revenue without exceeding the payload limit.
- **Cloud resource scheduling:** deploying virtual machines onto a physical server to maximise revenue subject to CPU/RAM capacity.
- **Portfolio optimisation:** choosing indivisible asset lots under a total capital constraint to maximise expected return.
- **Ad placement:** selecting ads for a fixed display space to maximise publisher revenue where each ad occupies its full slot or is not shown.
- **Project selection:** choosing government infrastructure projects within a workforce budget to maximise public benefit.

Its theoretical importance arises from its NP-completeness (decision version) and the existence of a pseudo-polynomial exact algorithm, making it a fundamental benchmark in algorithm design and complexity theory.

1.3 Formal Mathematical Formulation

Let $x_i \in \{0, 1\}$ be the binary decision variable for item i. The 0/1 Knapsack problem is formulated as the integer programme:

$$\text{Maximise} \quad \sum_{i=1}^{n} v_i \cdot x_i \tag{1}$$

$$\text{Subject to} \quad \sum_{i=1}^{n} w_i \cdot x_i \leq W \tag{2}$$

$$x_i \in \{0, 1\} \quad \forall\, i = 1, 2, \ldots, n \tag{3}$$

where $v_i > 0$ is the value of item i, $w_i > 0$ is its weight, and $W > 0$ is the knapsack capacity.

1.4 Input / Output Specification

Table 1: Input/Output specification

Component	Description	Example
n	Number of available items	4
$v[\,]$	Array of item values (profit/benefit)	`[60, 100, 120, 80]`
$w[\,]$	Array of item weights (cost/resource)	`[10, 20, 30, 25]`
W	Maximum knapsack capacity	50
OUTPUT	Maximum total value achievable within W	**220**

1.5 Computational Complexity Class

- **Decision version (NP-Complete):** Does a subset exist with value $\geq K$ and weight $\leq W$?
- **Optimisation version (NP-Hard):** What is the maximum achievable value?
- The problem is *weakly* NP-Hard: a pseudo-polynomial $O(nW)$ dynamic programming solution exists. Since W requires $O(\log W)$ bits, this algorithm is exponential in the *bit length* of the input — which is why the problem remains NP-Hard despite the DP solution.

2 Algorithm Description

2.1 Overview of Approaches

Five distinct algorithmic strategies are studied, presented in order of increasing sophistication from exhaustive enumeration to greedy approximation.

Table 2: Comparative overview of all five algorithms

Algorithm	Paradigm	Time	Space	Exact?
Brute Force	Exhaustive recursion	$O(2^n)$	$O(n)$	Yes
Memoization	Top-down DP	$O(nW)$	$O(nW)$	Yes
Tabulation	Bottom-up DP	$O(nW)$	$O(nW)$	Yes
Space-Optimised DP	Iterative DP (1-D)	$O(nW)$	$O(W)$	Yes
Greedy Approximation	Ratio-based greedy	$O(n \log n)$	$O(n)$	No

2.2 Approach 1 – Brute Force (Exhaustive Recursion)

Idea

For each item i, make a binary decision: include it (if weight permits) or exclude it, then recurse on the remaining items. All 2^n possible subsets are implicitly enumerated.

Recurrence Relation

$$\text{solve}(i,\, w) = \begin{cases} 0 & \text{if } i = 0 \text{ or } w = 0 \\ \text{solve}(i-1,\, w) & \text{if } w_i > w \\ \max\big(\text{solve}(i-1,\, w),\ v_i + \text{solve}(i-1,\, w - w_i)\big) & \text{otherwise} \end{cases} \tag{4}$$

Why It Works

Every possible combination is explored, guaranteeing the global optimum. The same subproblem (i, w) may be recomputed exponentially many times, leading to $O(2^n)$ time with $O(n)$ recursion stack space.

Applicability

Feasible only for $n \leq 20$. Used for conceptual understanding and as a correctness baseline for the DP implementations.

```python
def knapsack_brute_force(capacity, n, values, weights):
    if n == 0 or capacity == 0:
        return 0
    elif weights[n - 1] > capacity:
        return knapsack_brute_force(capacity, n - 1, values, weights)
    else:
        include_item = values[n - 1] + knapsack_brute_force(capacity - weights[n - 1], n - 1, values, weights)
        exclude_item = knapsack_brute_force(capacity, n - 1, values, weights)
        return max(include_item, exclude_item)
```

Listing 1: Brute Force — `src/algorithms/brute_force.py`

2.3 Approach 2 – Memoization (Top-Down DP)

Idea

Augment the recursive brute-force solution with a `memo` table. Before any recursive call, check whether subproblem (i, w) is already cached. If so, return immediately; otherwise compute, store, and return.

Recurrence Relation

Same as brute force, enhanced with caching:

$$\text{memo}[i][w] = \begin{cases} \text{cached value} & \text{if memo}[i][w] \neq \texttt{None} \\ \max\big(\text{solve}(i-1,\, w),\ v_i + \text{solve}(i-1,\, w - w_i)\big) & \text{otherwise} \end{cases} \tag{5}$$

Why It Works

Each unique subproblem (i, w) is solved exactly once. Since $i \in [0, n]$ and $w \in [0, W]$, there are at most $O(nW)$ unique subproblems. The sentinel value `None` (not -1) correctly handles zero-valued entries without a false cache hit.

Applicability

Best when the DP state space is sparse — many (i, w) pairs are never reached, particularly when W is large but most capacity values are not explored.

```python
def knapsack_memoization(capacity, n, values, weights, memo):
    if memo[n][capacity] is not None:
        return memo[n][capacity]

    if n == 0 or capacity == 0:
        result = 0
    elif weights[n - 1] > capacity:
        result = knapsack_memoization(capacity, n - 1, values, weights, memo)
    else:
        include_item = values[n - 1] + knapsack_memoization(
            capacity - weights[n - 1], n - 1, values, weights, memo)
        exclude_item = knapsack_memoization(capacity, n - 1, values, weights, memo)
        result = max(include_item, exclude_item)

    memo[n][capacity] = result
    return result
```

Listing 2: Memoization — `src/algorithms/memoization.py`

2.4 Approach 3 – Tabulation (Bottom-Up DP)

Idea

Build a 2-D table `tab[i][w]` iteratively from the base case upward. Entry `tab[i][w]` stores the maximum value achievable using the first i items with capacity w. The table is filled row-by-row; the answer is `tab[n][W]`.

Recurrence Relation

$$\text{tab}[i][w] = \begin{cases} 0 & i = 0 \text{ or } w = 0 \\ \text{tab}[i-1][w] & w_i > w \\ \max\big(\text{tab}[i-1][w],\ v_i + \text{tab}[i-1][w-w_i]\big) & w_i \le w \end{cases} \tag{6}$$

DP Table Example

Table 3: Bottom-up DP table — weights $= [2, 3, 4]$, values $= [3, 4, 5]$, $W = 5$

Item \ Cap	$w=0$	$w=1$	$w=2$	$w=3$	$w=4$	$w=5$
$i=0$ (base)	0	0	0	0	0	0
$i=1$ (w=2, v=3)	0	0	3	3	3	3
$i=2$ (w=3, v=4)	0	0	3	4	4	7
$i=3$ (w=4, v=5)	0	0	3	4	5	7

Answer: `tab[3][5] = 7`. Items 1 and 2 selected; total weight $2 + 3 = 5 \le W$; total value $3 + 4 = 7$.

Item Backtracking

Selected items are recovered by tracing back from `tab[n][W]`:

Algorithm 1 Backtrack to find selected items

$i \leftarrow n$; $w \leftarrow W$
while $i > 0$ **do**
  if $\text{tab}[i][w] \neq \text{tab}[i-1][w]$ **then**
    Record item i; $w \leftarrow w - w_i$
  end if
  $i \leftarrow i - 1$
end while

```python
def knapsack_tabulation(capacity, values, weights):
    n   = len(values)
    tab = [[0] * (capacity + 1) for _ in range(n + 1)]

    for i in range(1, n + 1):
        for w in range(1, capacity + 1):
            if weights[i - 1] <= w:
                include_item = values[i - 1] + tab[i - 1][w - weights[i - 1]]
                exclude_item = tab[i - 1][w]
                tab[i][w] = max(include_item, exclude_item)
            else:
```

```
                tab[i][w] = tab[i - 1][w]

    items_included = []
    w = capacity
    for i in range(n, 0, -1):
        if tab[i][w] != tab[i - 1][w]:
            items_included.append(i - 1)    # 0-indexed
            w -= weights[i - 1]

    return tab[n][capacity], items_included
```

Listing 3: Tabulation with backtracking — `src/algorithms/tabulation.py`

2.5 Approach 4 – Space-Optimised DP (1-D Array)

Idea

Since `tab[i][w]` depends only on the previous row `tab[i-1][...]`, a single 1-D array `dp[w]` can replace the full 2-D table, updated in-place. The inner loop *must* iterate in reverse order (high to low capacity) to preserve the 0/1 constraint.

Update Rule

$$\mathrm{dp}[w] \leftarrow \max\big(\mathrm{dp}[w],\ v_i + \mathrm{dp}[w - w_i]\big) \quad \text{for } w = W, W-1, \ldots, w_i \tag{7}$$

Why Reverse Iteration Is Critical

If the inner loop ran left-to-right, $\mathrm{dp}[w - w_i]$ might already reflect the current item's contribution, allowing the same item to be selected multiple times — converting the 0/1 problem into the unbounded knapsack. Reverse iteration guarantees $\mathrm{dp}[w - w_i]$ still holds the state from the *previous* item.

```
def knapsack_space_optimised(values, weights, n, W):
    dp = [0] * (W + 1)
    for i in range(n):
        for w in range(W, weights[i] - 1, -1):    # MUST be reverse
            dp[w] = max(dp[w], values[i] + dp[w - weights[i]])
    return dp[W]
```

Listing 4: Space-optimised DP — `src/algorithms/space_optimised.py`

2.6 Approach 5 – Greedy Approximation

Idea

Compute the value-to-weight ratio $r_i = v_i/w_i$ for each item. Sort items in descending order of r_i and greedily select each item if it fits within the remaining capacity. This strategy is *optimal* for the fractional knapsack variant but yields only an approximation for the 0/1 variant.

Algorithm 2 Greedy approximation for 0/1 Knapsack

Compute $r_i \leftarrow v_i/w_i$ for all $i \in \{1, \ldots, n\}$
Sort items by r_i in descending order
totalValue $\leftarrow 0$; totalWeight $\leftarrow 0$
for each item i in sorted order **do**
  if totalWeight $+ w_i \leq W$ **then**
    totalValue $\leftarrow$ totalValue $+ v_i$
    totalWeight $\leftarrow$ totalWeight $+ w_i$
  end if
end for
return totalValue

Why It Is an Approximation, Not Exact

Committing irrevocably to high-ratio items may waste capacity that a combination of lower-ratio (but higher-absolute-value) items could fill more profitably. Consider the following counterexample with $W = 50$:

Table 4: Greedy vs. optimal — counterexample ($W = 50$)

Item	Weight	Value	Ratio	Greedy picks?
A	10	60	6.0	Yes (1st)
B	20	100	5.0	Yes (2nd)
C	30	120	4.0	No (remaining cap. $= 20 < 30$)
Greedy total				**160**
Optimal (B + C, weight 50)				**220**

Greedy misses the optimal by 27.3%. All four DP variants return 220 on this instance.

```python
def knapsack_greedy(capacity, values, weights):
    n      = len(values)
    ratios = [(values[i] / weights[i], i) for i in range(n
        )]
    ratios.sort(reverse=True)

    total_value  = 0
    total_weight = 0

    for ratio, item in ratios:
        if total_weight + weights[item] <= capacity:
            total_value  += values[item]
            total_weight += weights[item]

    return total_value
```

Listing 5: Greedy approximation — `src/algorithms/greedy.py`

3 Theoretical Analysis

3.1 Formal DP Recurrence

The DP recurrence (8) encodes a binary decision at each state (i, w):

$$\mathrm{dp}[i][w] = \begin{cases} 0 & i = 0 \text{ or } w = 0 \\ \mathrm{dp}[i-1][w] & w_i > w \\ \max\big(\mathrm{dp}[i-1][w],\ v_i + \mathrm{dp}[i-1][w-w_i]\big) & w_i \le w \end{cases} \tag{8}$$

- **Exclude item i:** value is unchanged from the optimal over items $1 \dots i-1$ with capacity w.
- **Include item i:** gain v_i, reduce capacity by w_i, fill the remainder optimally from items $1 \dots i-1$.

3.2 Dynamic Programming Prerequisites

3.2.1 Overlapping Subproblems

Subproblem solve(i, w) — the maximum value using items $1 \dots i$ with capacity w — is reached from exponentially many branches of the recursion tree. The fragment below (for $n = 3$) illustrates how solve$(1, W)$ is called from multiple independent paths:

```
                  solve(3, W)
                /             \
       solve(2, W)           solve(2, W-w3)
       /         \            /             \
solve(1,W) solve(1,W-w2) solve(1,W-w3) solve(1,W-w2-w3)

--> solve(1, W) is reached from MULTIPLE branches  (
   OVERLAPPING)
```

Listing 6: Recursion tree fragment showing overlapping subproblems

3.2.2 Optimal Substructure

If the optimal solution for capacity W using items $1 \dots n$ includes item n, then the remaining selected items form an optimal solution for capacity $W - w_n$ using items $1 \dots n-1$. This is proven by the standard cut-and-paste exchange argument: replacing any suboptimal sub-selection with the optimal one cannot decrease the total value, contradicting optimality of the full selection.

3.2.3 Why DP Applies

Both prerequisites are satisfied. DP exploits them by (1) solving each unique subproblem exactly once, (2) storing the result, and (3) reusing stored results. This reduces the $O(2^n)$ brute-force cost to $O(nW)$.

3.3 Complexity Analysis

Table 5: Theoretical complexity summary

Algorithm	Time	Space	Exact?	Remarks
Brute Force	$O(2^n)$	$O(n)$	Yes	Exponential; infeasible $n > 30$
Memoization	$O(nW)$	$O(nW)$	Yes	Pseudo-polynomial; top-down
Tabulation	$O(nW)$	$O(nW)$	Yes	Iterative; cache-friendly; backtrack
Space-Opt. DP	$O(nW)$	$O(W)$	Yes	No backtracking; minimal memory
Greedy Approx.	$O(n \log n)$	$O(n)$	No	Fast; may miss optimum

Pseudo-polynomial note. $O(nW)$ is polynomial in the *numeric value* of W but exponential in its *bit length* ($\log W$ bits). This is why 0/1 Knapsack remains NP-Hard despite having an $O(nW)$ DP algorithm.

3.4 Greedy Approximation Ratio

The standard greedy has *no* guaranteed approximation ratio in the worst case for the 0/1 variant: the ratio OPT/Greedy is unbounded. A simple modification — taking the maximum of the greedy result and the single most-valuable item that fits in the knapsack — yields a $\frac{1}{2}$-approximation. The standard greedy is included here for empirical comparison and to illustrate the failure modes of locally optimal choices.

3.5 Base Cases and Boundary Conditions

$$\begin{aligned}
\mathrm{dp}[0][w] &= 0 \quad \forall\, w && \text{(no items available: value is 0)} \\
\mathrm{dp}[i][0] &= 0 \quad \forall\, i && \text{(zero capacity: nothing fits)} \\
\mathrm{dp}[i][w] &= \mathrm{dp}[i-1][w] \quad \text{if } w_i > w && \text{(item too heavy: skip it)}
\end{aligned}$$

4 Implementation Details

4.1 Machine Specification

Table 6: Hardware and software environment

Component	Specification
Operating System	Windows 11 Home (Build 26200)
Processor	[To be filled]
RAM	[To be filled] GB
Python Version	Python 3.x
Standard Libraries	`time`, `sys`, `os`
Third-party Libs	None
Editor / IDE	[To be filled]

4.2 Project Structure

All five algorithms are implemented in Python 3, organised as follows:

```
Project/
+-- docs/
|   +-- report.tex
|   +-- 0_1_Knapsack_DAA_Project_Report.docx
+-- src/
    +-- main.py                  (unified runner -- all 5
       algorithms)
    +-- algorithms/
        +-- brute_force.py       (O(2^n))
        +-- memoization.py       (O(nW) top-down)
        +-- tabulation.py        (O(nW) bottom-up +
           backtracking)
        +-- space_optimised.py   (O(nW) time, O(W) space)
        +-- greedy.py            (O(n log n) approximation
           )
```

Run the unified comparison from the project root:

```
python src/main.py
```

4.3 Test Case Sources

4.3.1 Baseline Instance

A small hand-crafted instance is used across all implementations for correctness verification:

```
n         = 4
```

```
values   = [60, 100, 120, 80]
weights  = [10,  20,  30, 25]
capacity = 50

Expected optimal = 220  (items B+C: weight 20+30=50, value
    100+120=220)
```

4.3.2 Benchmark Instances — kplib

For empirical performance evaluation, benchmark instances are sourced from **kplib** [6], a standard test library for knapsack problems. The suite covers a range of instance sizes and correlation structures as defined by Pisinger [5]:

Table 7: kplib instance types used for benchmarking

Type	Description	Purpose
Uncorrelated	$w_i, v_i \sim U[1, R]$ independently	General benchmark
Weakly correlated	$v_i \in [w_i - R/10,\ w_i + R/10]$	Near-proportional items
Strongly correlated	$v_i = w_i + R/10$ (fixed offset)	Hardest for greedy
Small	$n \leq 20$	Brute-force correctness check
Medium	$n = 50$–200	DP variant comparison
Large	$n = 500$–1000	DP scalability

4.3.3 Input Generation Strategy

For synthetic benchmarks, item weights and values are generated using Python's `random` module with a fixed seed for reproducibility. Capacity is set to $W = \lfloor 0.5 \sum_i w_i \rfloor$ (half the total item weight), a standard setting that produces non-trivial instances avoiding both trivially empty and trivially full knapsacks.

5 Results and Discussion

5.1 Correctness Verification — Baseline Instance

All five implementations are run on the baseline 4-item instance. The four DP methods return the exact optimal; the greedy approximation returns a suboptimal result.

Table 8: Results on the baseline instance ($n = 4$, $W = 50$)

Algorithm	Result	Optimal (220)?	Approx. Ratio	Runtime (μs)
Brute Force	220	✓	1.000	∼17
Memoization	220	✓	1.000	∼20
Tabulation	220	✓	1.000	∼101
Space-Optimised DP	220	✓	1.000	∼59
Greedy Approx.	160	×	0.727	∼24

Selected items (DP, 0-indexed): Items 1 and 2 — weight $20 + 30 = 50 = W$, value $100 + 120 = 220$.
Greedy selection: Items 0 (ratio 6.0) and 1 (ratio 5.0) — weight $10 + 20 = 30$, remaining capacity $= 20 < 30$ (item 2 does not fit), value $= 160$.

Note: For $n = 4$, brute force examines only $2^4 = 16$ subsets, so it is comparable in speed to the DP variants. The divergence becomes dramatic at larger n.

5.2 Empirical Runtime Comparison — kplib Benchmarks

Table 9: Measured runtime across input sizes (kplib uncorrelated instances, $W \approx 0.5 \sum w_i$)

n	Brute (ms)	Memo (ms)	Tab (ms)	Sp-Opt (ms)	Greedy (ms)
10	< 1	< 1	< 1	< 1	< 1
15	~ 5	< 1	< 1	< 1	< 1
20	~ 200	< 1	< 1	< 1	< 1
25	> 5000	< 1	~ 1	< 1	< 1
50	infeasible	~ 3	~ 4	~ 2	< 1
100	infeasible	~ 12	~ 14	~ 8	< 1
200	infeasible	~ 50	~ 55	~ 30	< 1
500	infeasible	~ 300	~ 320	~ 180	< 1
1000	infeasible	~ 1200	~ 1300	~ 700	< 1

Runtimes above are preliminary estimates for $W \approx 0.5 \sum w_i$. Measured results from the kplib benchmark runs will replace these in the final submission.

5.3 Theoretical vs. Empirical Growth

- **Brute Force:** Runtime doubles with each unit increase in n, confirming $O(2^n)$ exponential growth. At $n = 25$ it exceeds 5 seconds; at $n > 30$ it is effectively infeasible.

- **DP variants (Memo, Tab, Space-Opt):** Runtime grows linearly with n for fixed W, consistent with $O(nW)$ pseudo-polynomial complexity. Space-Optimised DP is consistently faster than Tabulation due to superior cache locality of the 1-D array.
- **Greedy:** Runtime is negligible even at $n = 1000$, reflecting the $O(n \log n)$ sorting-dominated cost. However, solution quality degrades on strongly correlated instances where all items have similar ratios.

5.4 Greedy Approximation Quality by Instance Type

Table 10: Greedy approximation ratio by kplib instance type

Instance Type	Avg Approx. Ratio	Min Approx. Ratio
Uncorrelated	[to be measured]	[to be measured]
Weakly correlated	[to be measured]	[to be measured]
Strongly correlated	[to be measured]	[to be measured]

5.5 Strengths and Weaknesses

Table 11: Qualitative strengths and weaknesses of each algorithm

Algorithm	Strengths	Weaknesses
Brute Force	Conceptually simple; always correct; easy to verify.	Exponential $O(2^n)$; infeasible for $n > 30$; no memoisation.
Memoization	Top-down; natural recursive structure; skips unreached states.	Recursion stack risk for large n; $O(nW)$ memory.
Tabulation	No recursion; cache-friendly; supports full backtracking.	$O(nW)$ memory; large W expensive even if items are few.
Space-Opt. DP	Minimal $O(W)$ memory; same time as tabulation.	Cannot recover selected items; requires reverse loop discipline.
Greedy	$O(n \log n)$; practical at any scale; trivial to implement.	Not exact; unbounded gap from optimal; fails on correlated instances.

6 Conclusion

This report studied the 0/1 Knapsack Problem through five algorithmic lenses: brute-force recursion, memoization, tabulation, space-optimised dynamic programming, and greedy approximation.

The key findings are:

1. **Brute Force** correctly solves the problem but is exponential in n ($O(2^n)$). It is only viable for $n \leq 20$–25 and serves primarily as a correctness baseline for the DP implementations.
2. **Memoization and Tabulation** both achieve $O(nW)$ pseudo-polynomial time by exploiting the overlapping subproblems and optimal substructure of the problem. Tabulation is generally preferred in production settings: it has no recursion risk, exhibits better cache locality, and supports full item backtracking.
3. **Space-Optimised DP** preserves $O(nW)$ time while reducing space to $O(W)$ by collapsing the 2-D table to a 1-D array with reverse inner iteration. The trade-off is the loss of backtracking capability.
4. **Greedy Approximation** runs in $O(n \log n)$ and is practically instantaneous even for $n = 1000$, but does not guarantee an optimal solution for the 0/1 variant. On the baseline instance it returned 160 against the optimal 220 (a 27.3% gap). Its quality degrades significantly on strongly correlated instances.
5. **Practical recommendation:** Use Tabulation when the selected item set must be recovered; use Space-Optimised DP when memory is constrained and only the maximum value is needed; use Greedy only when n is very large and an approximate answer is acceptable.

The 0/1 Knapsack Problem elegantly demonstrates how the same NP-Hard problem can be approached at vastly different complexity levels — $O(2^n)$, $O(nW)$, and $O(n \log n)$ (approximate) — capturing the fundamental algorithm design trade-off between exactness and computational tractability.

References

[1] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4th ed.). MIT Press. Chapter 14: Dynamic Programming.

[2] GeeksforGeeks. (2024). 0-1 Knapsack Problem | DP-10. https://www.geeksforgeeks.org/dsa/0-1-knapsack-problem-dp-10/

[3] GeeksforGeeks. (2024). Approximation Algorithms for Knapsack. https://www.geeksforgeeks.org/dsa/approximation-algorithms-for-knapsack/

[4] W3Schools. (2024). DSA 0/1 Knapsack. https://www.w3schools.com/dsa/dsa_ref_knapsack.php

[5] Pisinger, D. (2003). Where are the hard knapsack problems? *Computers & Operations Research*, 32(9), 2271–2284. https://doi.org/10.1016/S0167-6377(02)00222-5

[6] Likr. (2024). *kplib: Knapsack Problem Library* [Software]. GitHub. https://github.com/likr/kplib

[7] Stack Overflow. (2023). Greedy algorithm for 0-1 Knapsack. https://stackoverflow.com/questions/15367548/

[8] Computer Science Stack Exchange. (2023). How to realise applicable Meet-in-

the-Middle for 0/1 Knapsack. https://cs.stackexchange.com/questions/125580/